```
])

utm_coords = transform_coordinates(local_coords, transform_matrix)
print(utm_coords)
```

#1 Create the transformation matrix
#2 Add a column of ones to make the coordinates homogeneous
#3 Apply the transformation as a matrix multiplication
#4 Return just the easting and northing (drop the homogeneous coordinate)
#5 UTM coordinates of the origin point
#6 Rotation angle in degrees of the site offset from true North
#7 The array of local coordinates

Figure 2.15 illustrates the coordinate transformation performed in listing 2.8, showing the original local coordinates in the 0,0 origin based reference grid, and then the transformed coordinates in the UTM Zone 36S grid, assuming that the site is offset by 30 degrees from true North.

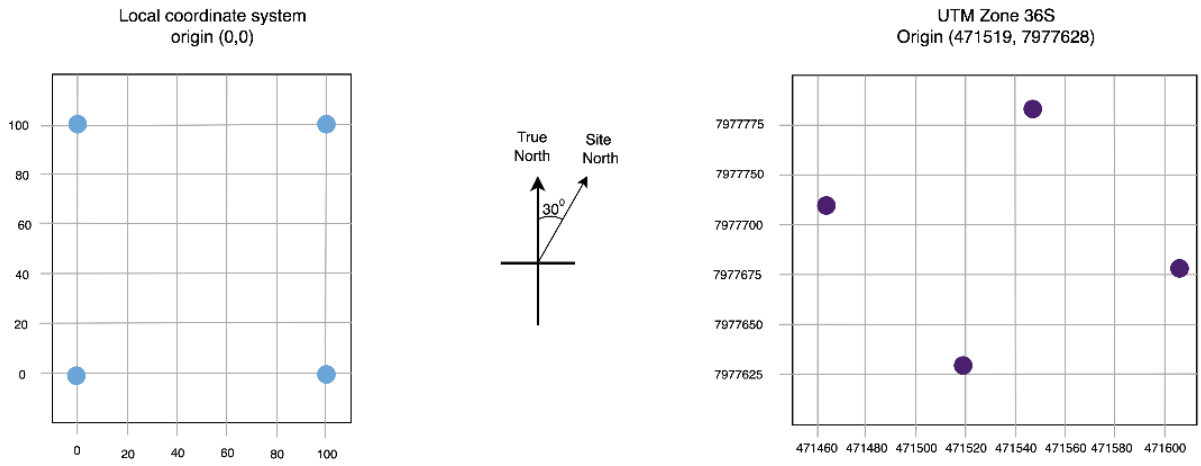


Figure 2.15 Visualization of the output of the code in listing 2.8 that converts local coordinates to UTM coordinates in zone 36S (EPSG:32736) with an origin point at 471519, 7977628 and a site offset of 30 degrees from true North.

While it is important to understand the mathematics of how coordinate transformations work, in practical digital twin applications, you can leverage higher level libraries to perform coordinate transforms for you. Listing 2.8 shows the use of the Python version of the PROJ library (https://proj.org), a popular library for geospatial coordinate transformation to convert between UTM coodinates and WGS84 with a single line of code.